



Searching (Pencarian)

Linear Search • Binary Search • Analisis Kompleksitas

Algoritma Pemrograman • IF25-11001 • 2 SKS Teori

Institut Teknologi Sumatera — Teknik Informatika

Review Pertemuan 10

Array 2D dan String — pondasi untuk searching

Array 2D = Tabel Baris × Kolom

`mat[M][N]`, akses `mat[baris][kolom]`, nested loop traversal, $O(M \times N)$.

String = Array of Char

Null terminator `\0`, `LENGTH`, `CONCAT`, `COMPARE`, `SUBSTRING` — semua $O(n)$.

Hari ini: MENCARI data dalam array!

Linear Search (brute force) vs Binary Search (divide dan conquer) — kapan pakai yang mana?

Capaian Pembelajaran Pertemuan 11

Sub-CPMK 11.1

C2 – Menjelaskan

Menjelaskan konsep pencarian data dan perbedaan Linear Search vs Binary Search

CPMK0609

Sub-CPMK 11.2

C3 – Menerapkan

Melakukan trace Linear Search step-by-step dan menganalisis kompleksitas $O(n)$

CPMK0609

Sub-CPMK 11.3

C3 – Menerapkan

Melakukan trace Binary Search dan menganalisis mengapa memerlukan array terurut ($O(\log n)$)

CPMK0609

Outline Pertemuan Hari Ini

	00-10	Review dan Hook	Recap array + motivasi searching
	10-25	Linear Search	Pseudocode, trace, analisis $O(n)$
	25-45	Binary Search	Prasyarat terurut, pseudocode, trace
	45-55	Perbandingan	$O(n)$ vs $O(\log n)$, kapan pakai mana?
	55-70	Workshop Trace	Trace kedua algoritma di kertas
	70-80	Algorithm Detective	Bug: infinite loop & off-by-one
	80-90	PBL#2 Milestone 2	Tambahkan searching ke Data Analyzer
	90-100	Exit Ticket dan Tugas	Assessment + homework



Pertanyaan Pemantik

*"Kamu punya 1000 nama mahasiswa.
Bagaimana cara menemukan 'Budi' secepat mungkin?"*

*"Kamus berisi 100.000 kata.
Apakah kamu baca dari halaman 1 atau langsung buka tengah?"*

*"Kenapa Google bisa menemukan hasil pencarian
dari miliaran halaman dalam 0.5 detik?"*

Linear Search (Sequential Search)

Cari elemen satu-per-satu dari awal sampai akhir

Cari target = 17 dalam arr = {5, 12, 17, 3, 9}

[0]	[1]	[2]	[3]	[4]
5	12	17	3	9
✗	✗	✓		

Pseudocode Linear Search

```
FUNGSI linearSearch(arr, N, target)
  UNTUK i ← 0 SAMPAI N-1
    JIKA arr[i] == target MAKA
      RETURN i
  AKHIR JIKA
  AKHIR UNTUK
  RETURN -1
AKHIR FUNGSI
```

Karakteristik

- Array TIDAK perlu terurut
- Cek elemen satu per satu dari indeks 0
- Best case: $O(1)$ — elemen pertama
- Worst case: $O(n)$ — elemen terakhir/tidak ada
- Return -1 jika tidak ditemukan

Trace: Linear Search

arr = {5, 12, 17, 3, 9}, target = 17

```
linearSearch(arr, 5, 17)
```

```
UNTUK i ← 0 SAMPAI 4  
  JIKA arr[i] == 17 MAKA  
    RETURN i  
  AKHIR JIKA  
AKHIR UNTUK  
RETURN -1
```

i	arr[i]	arr[i]==17?	Aksi	i≤4?	Return
0	5	5==17? F	lanjut	T	
1	12	12==17? F	lanjut	T	
2	17	17==17? T	RETURN 2		2

Ditemukan di indeks 2 — hanya perlu 3 perbandingan!

Trace: target = 99 (TIDAK ADA)

i	arr[i]	arr[i]==99?	Aksi	i	arr[i]	arr[i]==99?	Aksi
0	5	F	lanjut	3	3	F	lanjut
1	12	F	lanjut	4	9	F	lanjut
2	17	F	lanjut			Loop selesai	RETURN -1

Tidak ditemukan → RETURN -1 — harus cek SEMUA 5 elemen = $O(n)$ worst case

Binary Search (Pencarian Biner)

Array HARUS terurut — bagi dua setiap langkah

Cari target = 17 dalam arr = {3, 5, 9, 12, 17, 21, 25} (TERURUT!)

[0]	[1]	[2]	[3]	[4]	[5]	[6]
3	5	9	12	17	21	25
low=0			mid=3			high=6

Pseudocode Binary Search

```
FUNGSI binarySearch(arr, N, target)
  low ← 0
  high ← N-1
  SELAMA low ≤ high
    mid ← (low+high) / 2
    JIKA arr[mid] == target MAKA
      RETURN mid
    JIKA arr[mid] < target MAKA
      low ← mid + 1
    SELAINNYA
      high ← mid - 1
  AKHIR SELAMA
  RETURN -1
AKHIR FUNGSI
```

Karakteristik

- PRASYARAT: array HARUS terurut!
- Bandingkan target dengan elemen TENGAH
- Jika target < mid: cari di KIRI
- Jika target > mid: cari di KANAN
- Setiap langkah eliminasi SETENGAH data
- Best case: $O(1)$ — elemen tengah
- Worst case: $O(\log n)$ — JAUH lebih cepat!

Trace: Binary Search

arr = {3, 5, 9, 12, 17, 21, 25}, target = 17

Step	low	high	mid	arr[mid]	arr[mid] vs 17	Aksi	Return
1	0	6	3	12	12 < 17	low $\leftarrow$ 3+1 = 4	
2	4	6	5	21	21 > 17	high $\leftarrow$ 5-1 = 4	
3	4	4	4	17	17 == 17	DITEMUKAN!	4

Ditemukan di indeks 4 — hanya 3 langkah! Linear Search butuh 5 langkah.

Visualisasi Eliminasi Setengah Data

Step 1:	[3 5 9 12 17 21 25]	mid=12 < 17 → buang KIRI	-3 elemen
Step 2:	[17 21 25]	mid=21 > 17 → buang KANAN	-1 elemen
Step 3:	[17]	mid=17 == 17 → DITEMUKAN!	

Perbandingan: Linear vs Binary Search

$O(n)$ vs $O(\log n)$ — perbedaannya DRASTIS!

Jumlah Data (n)	Linear $O(n)$	Binary $O(\log_2 n)$	Berapa kali lebih cepat?
10	10 langkah	4 langkah	2.5x
100	100 langkah	7 langkah	14x
1.000	1.000 langkah	10 langkah	100x
1.000.000	1.000.000 langkah	20 langkah	50.000x
1.000.000.000	1 miliar langkah	30 langkah	33 jutax

Kapan Pakai Yang Mana?

Linear Search

Array TIDAK terurut, array kecil ($n < 20$), cari satu kali saja

Binary Search

Array SUDAH terurut, array besar, pencarian berulang kali

Trade-off

Sorting dulu = $O(n \log n)$, tapi pencarian jadi $O(\log n)$. Worth it jika banyak pencarian!

Workshop: Trace Searching

Kerjakan di kertas — 10 menit

Soal 1: Linear Search

Diberikan:

$\text{arr} = \{8, 3, 15, 7, 22, 11, 4\}$

$N = 7$

Pertanyaan:

- a) Trace linear search untuk target = 11.
Buat trace table (kolom: i , $\text{arr}[i]$,
 $\text{arr}[i]==11?$, aksi).
Berapa perbandingan?
- b) Trace linear search untuk target = 99.
Berapa total perbandingan?
- c) Apa best case dan worst case
untuk array ini?

Soal 2: Binary Search

Diberikan (sudah terurut!):

$\text{arr} = \{2, 5, 8, 12, 16, 23, 38, 45, 56\}$

$N = 9$

Pertanyaan:

- a) Trace binary search target = 23.
Buat trace table (kolom: step, low,
high, mid, $\text{arr}[\text{mid}]$, aksi).
- b) Trace binary search target = 10.
Kapan loop berhenti? Return apa?
- c) Berapa MAKSIMUM langkah untuk
array 9 elemen? (Hint: $\log_2 9$)

Jawaban Workshop

Soal 1a: Linear Search target=11 dan Soal 2a: Binary Search target=23

i	arr[i]	==11?	Aksi
0	8	F	lanjut
1	3	F	lanjut
2	15	F	lanjut
3	7	F	lanjut
4	22	F	lanjut
5	11	T	RETURN 5

6 perbandingan — ditemukan di indeks 5

Step	low	high	mid	arr[mid]	Aksi
1	0	8	4	16	$16 < 23 \rightarrow \text{low}=5$
2	5	8	6	38	$38 > 23 \rightarrow \text{high}=5$
3	5	5	5	23	DITEMUKAN!

3 langkah — ditemukan di indeks 5. Maks = $\lceil \log_2 9 \rceil = 4$

Jawaban Lanjutan

Soal 1b: target=99, linear search harus cek SEMUA 7 elemen $\rightarrow$ 7 perbandingan, RETURN -1.

Soal 1c: Best case = $O(1)$ jika target di indeks 0. Worst case = $O(n) = 7$ langkah.

Soal 2b: target=10. Step1: mid=4, arr[4]=16 > 10 $\rightarrow$ high=3. Step2: mid=1, arr[1]=5 < 10 $\rightarrow$ low=2.

Step3: mid=2, arr[2]=8 < 10 $\rightarrow$ low=3. Step4: mid=3, arr[3]=12 > 10 $\rightarrow$ high=2.

low(3) > high(2) $\rightarrow$ loop berhenti, RETURN -1. Tidak ditemukan setelah 4 langkah.

Soal 2c: Maks langkah = $\lceil \log_2 9 \rceil = \lceil 3.17 \rceil = 4$ langkah (sesuai trace 2b!).



Algorithm Detective

Temukan bug pada searching berikut!

Kasus 1: Binary Search Tanpa Sort

```
arr ← {15, 3, 22, 8, 17}  
target ← 8  
hasil ← binarySearch(arr, 5, 8)
```

Expected: ditemukan indeks 3
Actual: TIDAK ditemukan! (return -1)

Bug: Array TIDAK terurut!
Binary search HANYA benar jika
array sudah diurutkan.

Fix: Sort dulu, BARU binary search.
Atau gunakan linear search.

Kasus 2: Infinite Loop Binary

```
SELAMA low ≤ high  
  mid ← (low+high) / 2  
  JIKA arr[mid] == target MAKA  
    return mid  
  SELAINNYA JIKA arr[mid] < target MAKA  
    low ← mid  
  SELAINNYA  
    high ← mid
```

Bug: low ← mid (bukan mid+1)
high ← mid (bukan mid-1)

Jika low=3, high=4, mid=(3+4)/2=3
low tetap 3 → INFINITE LOOP!

Fix:
low ← mid + 1
high ← mid - 1



Aturan Emas Searching

Hindari bug yang paling umum

1

Binary Search = array TERURUT

Jangan pernah pakai binary search pada array yang belum disort.
Hasilnya SALAH!

Cek: apakah sudah sorted?

2

low $\leftarrow$ mid+1, high $\leftarrow$ mid-1

Tanpa +1/-1, loop bisa infinite. Setiap iterasi harus memperkecil range.

SELALU tambah/kurang 1

3

Kondisi: SELAMA low $\leq$ high

Gunakan $\leq$ bukan $<$. Jika low==high, masih ada 1 elemen yang belum dicek!

SELAMA low $\leq$ high

4

Return -1 = TIDAK ditemukan

Selalu tangani kasus tidak ditemukan. Jangan asumsi target pasti ada di array.

Selalu cek if result == -1

PBL#2: Data Analyzer — Milestone 2

Tambahkan fitur searching ke Data Analyzer

Milestone 2: Fitur Pencarian (P11)

✓ Milestone 1 selesai (statistik array)

Tambahkan fitur pencarian ke Data Analyzer:

- User input: "Cari nilai X" → program cari di array
- Implementasi Linear Search (wajib)
- Implementasi Binary Search (wajib, sort dulu!)
- Output: posisi/indeks elemen, atau "Tidak ditemukan"
- Bandingkan jumlah langkah linear vs binary

Timeline PBL#2

P9-P10

Statistik array (total, rata, maks, min)

Milestone 1 ✓

P11

Searching: Linear dan Binary Search

Milestone 2 👉

P12

Sorting: Bubble dan Selection Sort → SUBMIT

Deadline!

Ringkasan Pertemuan 11

Linear Search = $O(n)$

Cek satu-per-satu. Tidak perlu array terurut. Sederhana tapi lambat untuk data besar.



Binary Search = $O(\log n)$

Bagi dua setiap langkah. HARUS array terurut. Sangat cepat untuk data besar.

1 miliar data = 30 langkah!

Binary search: $\log_2(10^9) \approx 30$. Linear search: 1 miliar langkah. Perbedaan LUAR BIASA.



Prasyarat Binary: SORTED

Jangan pernah pakai binary search tanpa sort dulu. Hasilnya pasti salah.

PBL#2 Milestone 2

Tambahkan fitur searching ke Data Analyzer. Deadline sorting di P12.



Konsep searching ini akan di-CODING di Praktikum P11 — siapkan pseudocode!

Exit Ticket — Pertemuan 11

10 soal • 15 menit • kerjakan di kertas

1

Apa perbedaan utama Linear Search dan Binary Search? (2 hal)

2

arr={4,9,2,7}. Trace linear search target=7. Berapa perbandingan?

3

Kenapa binary search HARUS menggunakan array terurut?

4

arr={3,8,15,22,31,40}. Trace binary search target=31.

5

Array 1024 elemen. Berapa MAKSIMUM langkah binary search?

6

Apa arti RETURN -1 pada fungsi searching?

7

Bug: $\text{low} \leftarrow \text{mid}$ (bukan $\text{mid}+1$). Apa yang terjadi? Mengapa?

8

arr={1,5,10,15,20}. Binary search target=7. Trace sampai berhenti.

9

Kapan sebaiknya pakai linear search daripada binary search?

10

Array 1.000.000 elemen. Bandingkan langkah linear vs binary.

Tugas Mingguan — Searching

Deadline: sebelum Pertemuan 12 • Kerjakan di kertas

Tugas A: Individual — Trace dan Analisis

Soal 1: arr = {14, 27, 3, 10, 35, 19, 42, 8, 31, 5}. N=10.

- (a) Trace linear search target=19. Berapa perbandingan?
- (b) Trace linear search target=50. Berapa perbandingan?

Soal 2: Urutkan array soal 1 (tuliskan hasil sort), lalu:

- (a) Trace binary search target=19.
- (b) Trace binary search target=20 (tidak ada).
- (c) Bandingkan jumlah langkah linear vs binary untuk kedua target.

Soal 3: Berapa MAKSIMUM langkah binary search untuk array 50 elemen?
Jelaskan rumus dan hitung.

Tugas B: Kelompok — PBL#2 Milestone 2

Tambahkan fitur searching ke Data Analyzer: user input target → linear search + binary search.
Siapkan pseudocode kedua algoritma dan trace table untuk data contoh minimal 8 elemen.

Bobot Tugas A: Trace Table (40%) • Analisis Kompleksitas (30%) • Perbandingan (30%)

Referensi Pertemuan 11

Buku

1. Deitel — C++ How to Program, Bab 7.7: Searching Arrays
2. Gaddis — Starting Out with C++, Bab 8: Searching
3. Cormen (CLRS) — Introduction to Algorithms, Bab 2: Binary Search

Online

1. visualgo.net/sorting — Visualisasi searching interaktif
2. cs50.harvard.edu — Lecture 3: Algorithms (Searching)
3. [geeksforgeeks.org/binary-search](https://www.geeksforgeeks.org/binary-search) — Binary Search Tutorial



Terima Kasih

IF25-11001 Algoritma Pemrograman

Pertemuan 11: Searching (Pencarian)

Program Studi Teknik Informatika
Institut Teknologi Sumatera
2026